# Module 6: FSDP and ZeRO

## *Trading Redundant Memory for Collective Communication*

**BRAIN team**

IMT Atlantique · January 2027

IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

BRAIN

# Lecture map

1. From replication to sharding
2. Execution and wrapping
3. Checkpointing and fair comparison

**Guiding question**

Which persistent states can be sharded, when must they be materialized, and what does that cost on the critical path?

# What should be clear by the end?

1. Account for DDP's replicated model-state memory.
2. Explain ZeRO stages as progressive sharding.
3. Trace one FSDP unit through forward and backward.
4. Choose wrapping, precision, and checkpoint strategies from evidence.

PART 1

# From replication to sharding

1. From replication to sharding
2. Execution and wrapping
3. Checkpointing and fair comparison

# DDP replicates persistent state on every rank

| State | bytes / param. | DDP placement |
|---|---|---|
| BF16 parameters | 2 | replicated |
| BF16 gradients | 2 | replicated |
| FP32 master copy | 4 | replicated |
| FP32 Adam moment $m$ | 4 | replicated |
| FP32 Adam moment $v$ | 4 | replicated |
| total model state | about 16 | on every GPU |

**The scaling limit**

More GPUs improve compute throughput but do not reduce DDP's replicated state.

# ZeRO removes one class of redundancy at a time

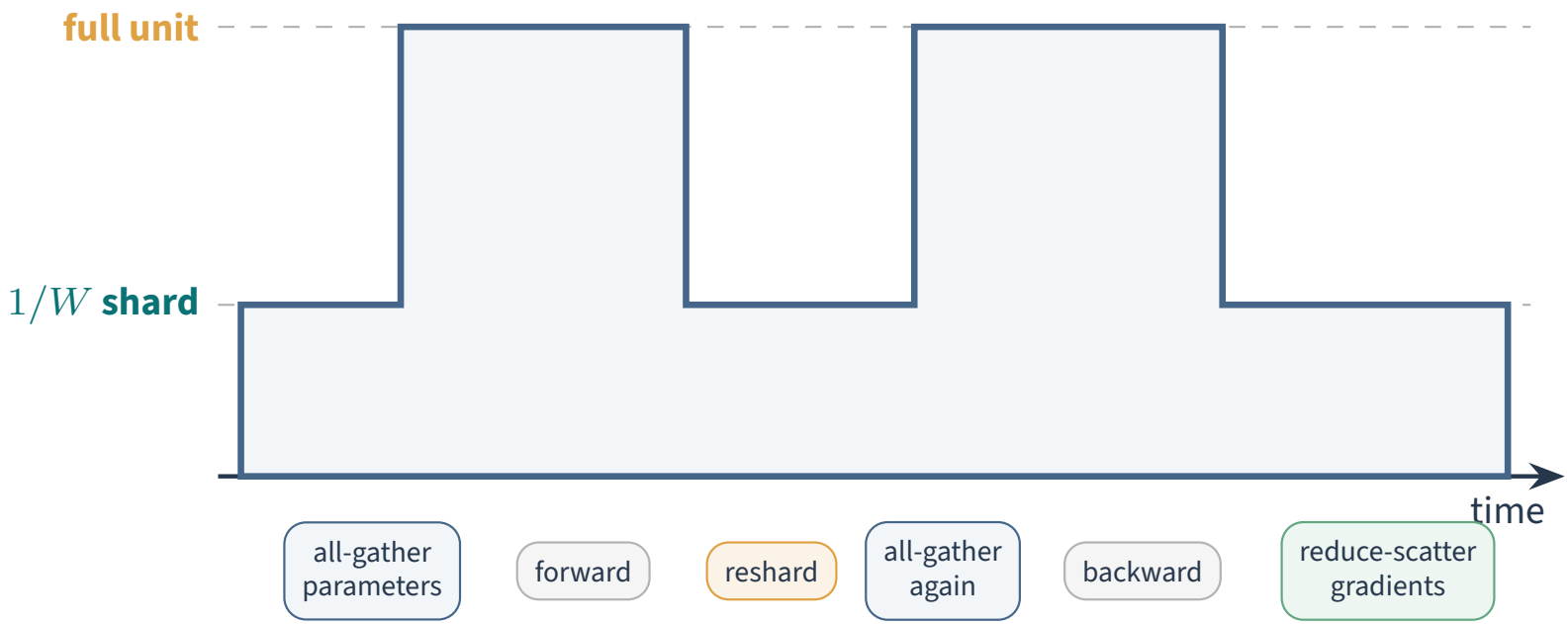
| Method | optimizer state | gradients | parameters |
|---|---|---|---|
| DDP | replicated | replicated | replicated |
| ZeRO-1 | sharded | replicated | replicated |
| ZeRO-2 | sharded | sharded | replicated |
| ZeRO-3 / FSDP | sharded | sharded | sharded |

## Idealized per-rank model state

With full sharding and world size $W$, persistent model-state memory approaches $1/W$ of the unsharded total, before activations, buffers, and temporary materialization.

Source: Hugging Face Ultra-Scale Playbook and ZeRO

# Sharding changes when full parameters exist




Full parameters are transient around compute; the local shard is persistent.

PART 2

# Execution and wrapping

1. From replication to sharding
2. Execution and wrapping
3. Checkpointing and fair comparison

# One FSDP unit is a materialization boundary

### Large unit

fewer collectives
larger peak all-gather
less opportunity for overlap
coarser memory release

### Tiny units

low peak materialization
many latency-dominated collectives
more metadata and scheduling overhead

Wrapping each Transformer block is a common starting point because it aligns materialization with a natural compute unit.

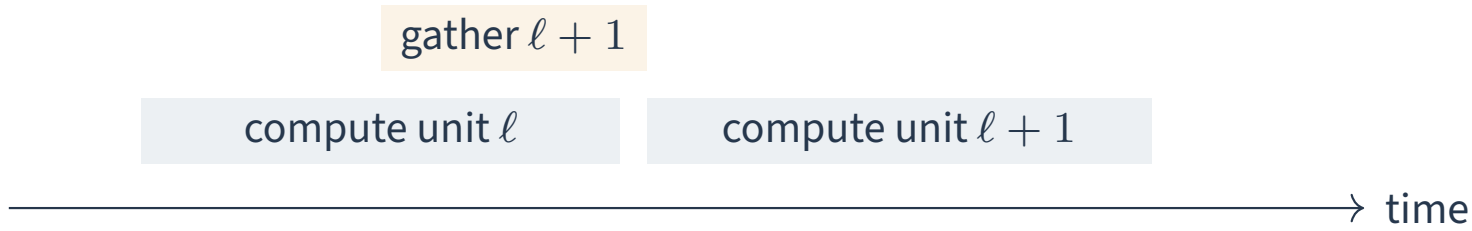
# Communication volume and exposure are different

- **Volume**: total bytes transferred by all-gathers and reduce-scatters.
- **Exposure**: communication time that remains visible after overlap with compute.
- **Frequency**: number of collectives and their message sizes.

**Why this matters**

Two wrapping policies can move similar bytes but have very different step times because one creates many small latency-bound collectives or a longer exposed tail.

# Prefetching tries to move the next unit early




**Constraint**

Prefetching improves overlap but raises transient memory because current and next full units may coexist.

# FSDP does not shard activation memory automatically

Full sharding reduces persistent parameter, gradient, and optimizer-state redundancy.

**Still scales with batch and sequence**

saved activations
attention workspaces
temporary full parameters
communication buffers

Combine FSDP with activation checkpointing when activations, not model state, dominate the peak.

# Mixed precision has three separate decisions

| Quantity | Common choice | Main trade-off |
|---|---|---|
| parameter compute dtype | BF16 | speed and temporary memory |
| gradient reduction dtype | BF16 or FP32 | bandwidth versus numerical range |
| optimizer state dtype | FP32 | stable moments at persistent cost |

**Experimental rule**

When comparing DDP and FSDP, align precision policy. Otherwise, memory and numerical differences have multiple causes.

# Checkpoint: wrapping policy

## Think, then compare

A run fits in memory but launches hundreds of tiny all-gathers and scales poorly. What wrapping change would you test, and which memory and timing quantities could regress?

PART 3

# Checkpointing and fair comparison

1. From replication to sharding
2. Execution and wrapping
3. Checkpointing and fair comparison

# A distributed checkpoint is a data layout

**Full**

easy to consume
expensive to gather
rank 0 memory pressure

**Sharded**

scalable writes
world-size coupling
unless resharded

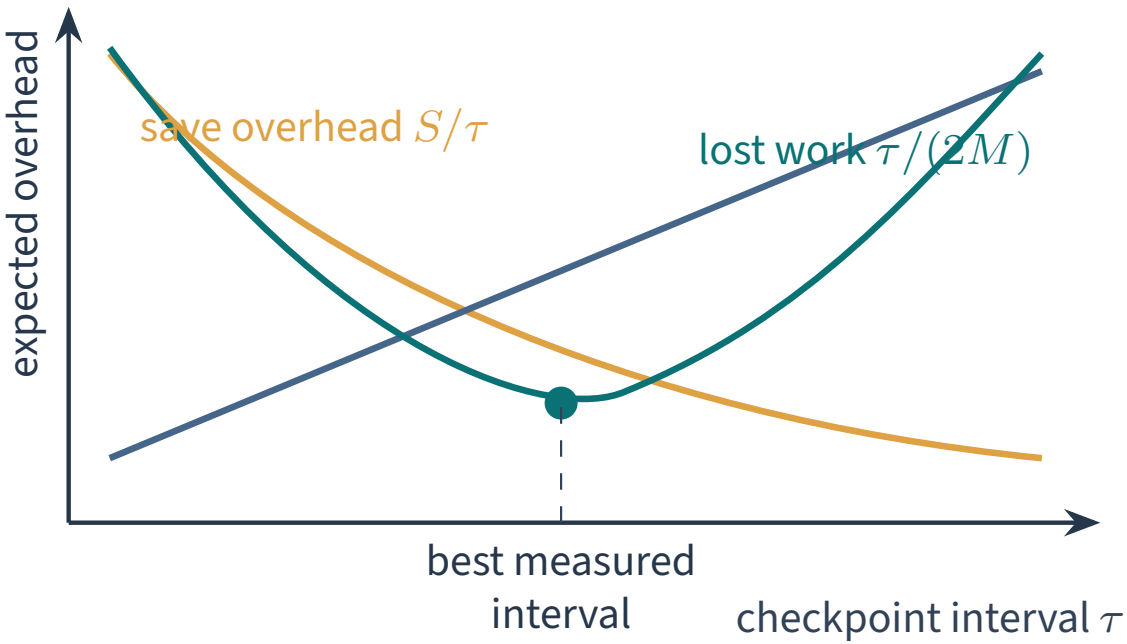
**Distributed format**

parallel I/O
metadata and
recovery complexity

**Completion protocol**

Publish a checkpoint only after every shard and its metadata are durable. A directory that exists is not proof of a complete save.

# Checkpoint cadence and expected overhead




$$\text{rate} \approx \frac{S}{\tau} + \frac{\tau}{2M} + \frac{L}{M}$$

$M$: mean time between interruptions

$S$: exposed save time

$L$: recovery load time

Measure durable save completion and restart time; file creation alone does not define $S$ or $L$.

Source: MVA Training and Deploying Large-Scale Models, Lecture 4 (2026)

# A DDP–FSDP comparison needs a fixed contract

Match model, global token batch, optimizer, schedule, precision, data order, checkpoint cadence, and evaluation.

**Correctness evidence**

first-update comparison
loss trajectory
resume equivalence

**Systems evidence**

peak allocated memory
tokens per second
collective exposure
checkpoint time and size

# Choose FSDP when memory is the binding constraint

- Prefer DDP when the model fits comfortably and communication simplicity wins.
- Prefer FSDP when replicated state blocks model size, batch size, or context.
- Tune wrapping and prefetch from traces, not from a universal recipe.
- Treat checkpoint load and resharding as part of the production design.

# Checkpoint: memory prediction

## Think, then compare

Estimate idealized model-state memory per rank for $P$ parameters, 16 bytes per parameter, and world size $W$. Then name three reasons the measured peak will be higher.

# The durable ideas

1. ZeRO stages progressively shard optimizer state, gradients, and parameters.
2. FSDP materializes one wrapped unit through all-gather and returns sharded gradients.
3. Wrapping controls peak memory, collective size, frequency, and overlap.
4. Checkpoint format and a matched DDP baseline are part of the system claim.